

Lab Report No. 04

Experiment Name

Implementation of Stack-Based Boundary Filling Algorithm Using OpenGL.

Objective

To implement the Stack-Based Boundary Filling Algorithm and fill a closed polygon using OpenGL.

Theory/Introduction

Boundary Fill Algorithm is a seed-fill algorithm used in computer graphics to fill the interior of a closed shape.

The algorithm starts from a seed point inside the boundary and fills neighboring pixels until the boundary color is reached.

In the **Stack-Based Boundary Fill Algorithm**, a stack is used instead of recursion. This avoids recursion depth limitations and is more efficient for larger areas.

Algorithm/Procedure

1. Create a closed polygon using line drawing.
 2. Select a seed point inside the polygon.
 3. Push the seed point onto a stack.
 4. Repeat until the stack becomes empty:
 5. Pop a point from the stack.
 6. If the point is neither a boundary pixel nor already filled:
 - Fill the point.
 - Push its four neighboring pixels (up, down, left, right) onto the stack.
 7. Continue until the entire interior region is filled.
 8. Display the filled polygon.
-

Source Code

```
from OpenGL.GL import *  
from OpenGL.GLUT import *  
from OpenGL.GLU import *
```

```

SIZE = 500

grid = [[0 for _ in range(SIZE)] for _ in range(SIZE)]
filled_points = []

def DDA(x1, y1, x2, y2):
    points = []

    dx = x2 - x1
    dy = y2 - y1

    steps = max(abs(dx), abs(dy))

    x_inc = dx / steps
    y_inc = dy / steps

    x = x1
    y = y1

    for i in range(steps + 1):
        points.append((round(x), round(y)))
        x += x_inc
        y += y_inc

    return points

def draw_boundary(points):
    for x, y in points:
        if 0 <= x < SIZE and 0 <= y < SIZE:
            grid[y][x] = 1

def boundary_fill_stack(seed_x, seed_y):
    stack = [(seed_x, seed_y)]

    while stack:
        x, y = stack.pop()

        if x < 0 or x >= SIZE or y < 0 or y >= SIZE:
            continue

        if grid[y][x] == 1 or grid[y][x] == 2:
            continue

        grid[y][x] = 2
        filled_points.append((x, y))

        stack.append((x + 1, y))
        stack.append((x - 1, y))
        stack.append((x, y + 1))
        stack.append((x, y - 1))

```

```

def reset_grid():
    filled_points.clear()

    for y in range(SIZE):
        for x in range(SIZE):
            grid[y][x] = 0

def display():
    glClear(GL_COLOR_BUFFER_BIT)

    reset_grid()

    A = (100, 100)
    B = (300, 120)
    C = (180, 300)

    edge1 = DDA(A[0], A[1], B[0], B[1])
    edge2 = DDA(B[0], B[1], C[0], C[1])
    edge3 = DDA(C[0], C[1], A[0], A[1])

    boundary_points = edge1 + edge2 + edge3

    draw_boundary(boundary_points)

    glColor3f(1, 1, 1)
    glBegin(GL_POINTS)
    for x, y in boundary_points:
        glVertex2i(x, y)
    glEnd()

    seed_x = (A[0] + B[0] + C[0]) // 3
    seed_y = (A[1] + B[1] + C[1]) // 3

    boundary_fill_stack(seed_x, seed_y)

    glColor3f(1, 0, 0)
    glBegin(GL_POINTS)
    for x, y in filled_points:
        glVertex2i(x, y)
    glEnd()

    glutSwapBuffers()

glutInit()
glutInitDisplayMode(GLUT_RGBA | GLUT_DOUBLE)
glutInitWindowSize(500, 500)

glutCreateWindow(b"Stack Based Boundary Fill")

glClearColor(0, 0, 0, 1)

```

```
gluOrtho2D(0, 500, 0, 500)

glutDisplayFunc(display)

glutMainLoop()
```

Sample Input

Closed Triangle:

- A = (100,100)
- B = (300,120)
- C = (180,300)

Seed Point:

- (193,173)
-

Sample Output

A white triangular boundary is displayed and its interior is filled with red color using the Stack-Based Boundary Fill Algorithm.

Result

The Stack-Based Boundary Filling Algorithm was successfully implemented. The selected polygon was filled correctly from the seed point without crossing the boundary.

Conclusion

The experiment was completed successfully. A stack was used to fill the interior region of a closed polygon. The Stack-Based Boundary Fill Algorithm efficiently filled the area while avoiding recursion-related limitations.